

MLOps on GCP for ARIMA Model using uWSGI Flask

Business Objective

MLOps is a set of practices that aims to deploy and maintain machine learning models in production reliably and efficiently. Today, many organizations are turning towards machine learning and Artificial intelligence. AI-based applications promise to deliver new levels of competitiveness, intelligence, and automation for businesses. MLOps is a means of continuous delivery and deployment of these machine learning models. Practicing MLOps means that you advocate for automation and monitoring at all steps of ML system construction, including integration, testing, releasing, deployment, and infrastructure management.

We aim to create the MLOps project by using the google cloud platform (GCP). Here, GCP is used as a cloud provider. But first, we would suggest you go through the former project [Time Series Forecasting Project-Building ARIMA Model in Python](#) before starting with this project.

Aim

To create an end-to-end machine learning development process to design, build and manage reproducible, testable, and evolvable machine learning models using Google Cloud Platform (GCP) for the Time Series ARIMA Project.

Tech stack

- Language - Python
- Services - GCP, uWSGI, Flask, Kubernetes, Docker

Approach

Steps:

- Clone the repository
- Place your model file inside the `output` folder

Once you made the changes, create a new repository and commit the changes. From here on, this will be your source repository. Proceed with the below steps

CLOUD BUILD TRIGGER

- In your GCP console, create a new cloud build trigger.

- Point the trigger to your source repository

GOOGLE KUBERNETES ENGINE (GKE)

- From the console launch a Kubernetes cluster
- Connect to the cluster and create the following two files
- deployment.yaml
- service.yaml
- Copy the code for both files from "Kubernetes Files" folder in cloned repository
- Execute the following commands
 - `kubectl apply -f deployment.yaml`
 - `kubectl apply -f service.yaml`
- Get the name of the deployment with the following command
 - `kubectl get deployments`

CLOUD PUB/SUB

- Create a Pub/Sub topic with the name `cloud-build`
- Provide a subscription for the topic, which is to trigger a cloud function

CLOUD FUNCTIONS

- From Pub/Sub console launch the cloud function window
- Provide the following Environment variables through the GUI console
 - `PROJECT` (project name)
 - `ZONE` (Region in which in the project is deployed ex.uscentral-1)
 - `CLUSTER` (Name of the kubernetes cluster created earlier)
 - `DEPLOYMENT` (Name of the deployment inside the kubernetes cluster)
- Copy the program code and requirements.txt files for the cloud function from cloud-function-trigger folder
- Configure the Entrypoint for the cloud function as `onNewImage`
- Deploy the function

After successful deployment, make a commit to source repository and the following will happen in sequence * Cloud Build will push message to Pub/Sub upon successful build * Pub/Sub will trigger the cloud function * Cloud function will deploy the new image on Kubernetes * To test the deployment, check the logs on kubernetes cluster using the following command * `kubectl get pods` * `kubectl logs <pod name>` * The deployment will reflect in the logs as well as in the endpoints

Folder Structure Info:

1. Input - Data-chillers.csv
2. Kubernetes files - this folder contains all the required files to trigger Kubernetes
3. MLPipeline - this folder contains all the functions put into different python files
4. Notebook - the time series arima notebook (seen in the former project)
5. Output- the model that is saved in the (.pkl format)
6. __init__.py - required empty file
7. Dockerfile - The docker image
8. Engine - File where the MLPipeline files are called
9. Main.py- file to host Flask API
10. Readme - explains the entire approach /steps
11. Requirements.txt - all the required libraries
12. UWSGI.ini - uWSGI configuration file

Project Takeaways

1. Introduction to MLOps
2. MLOps architecture in Google Cloud Platform (GCP)
3. Understanding of flask and uWSGI model files
4. Understanding and building of Docker images
5. Understanding of Kubernetes architecture
6. Various components on GCP
7. Learn how to create a cloud repository in GCP
8. Learn how to clone the git repository with the source repository
9. Learn how to commit changes in the source repository
10. Cloud build component and create a trigger
11. Understand the Pub/Sub component
12. Cloud Shell editor
13. Flask deployment
14. Understanding the cloud function
15. Kubernetes deployment